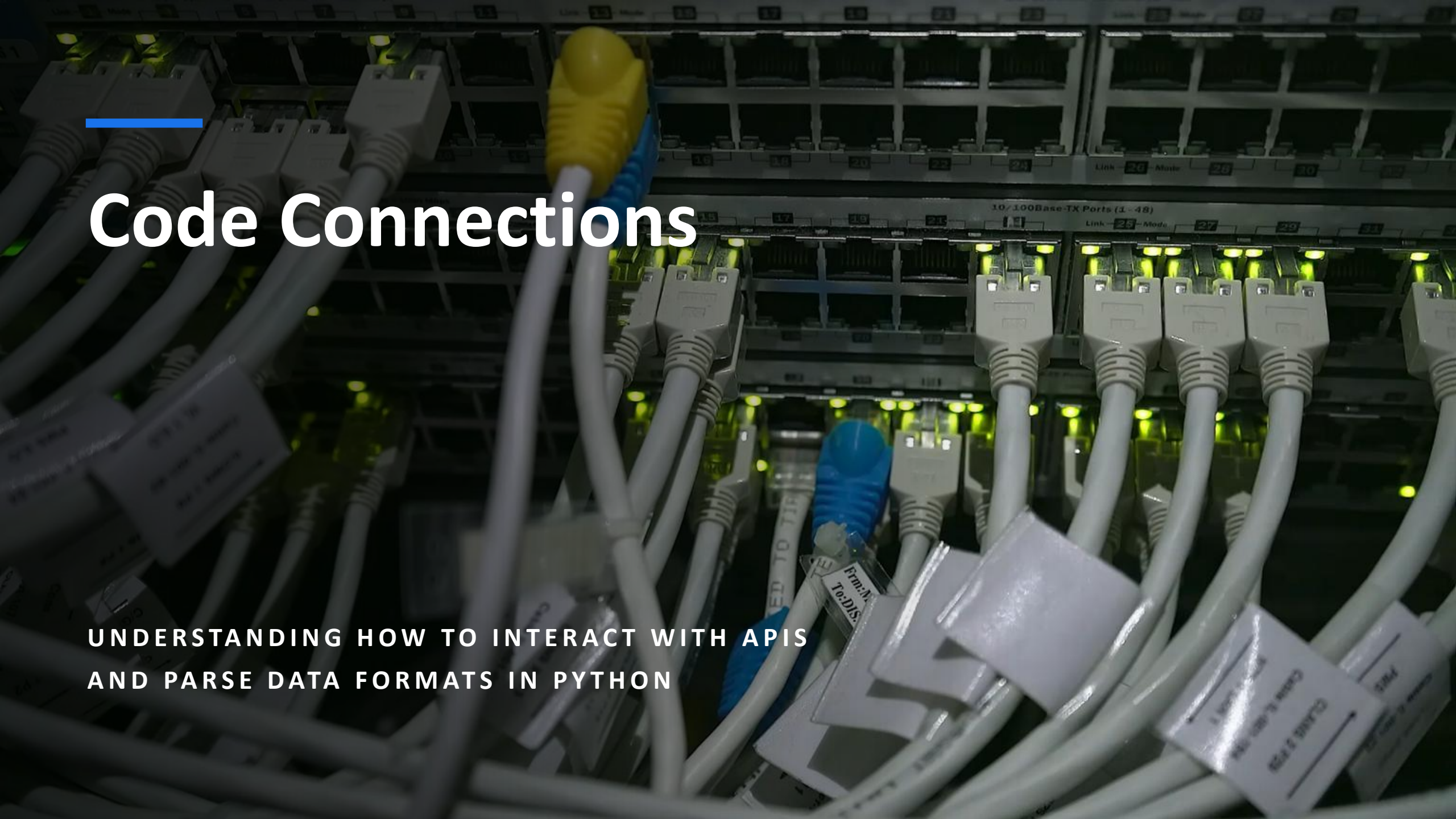




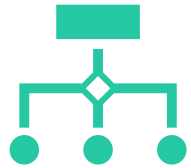
Code Connections

UNDERSTANDING HOW TO INTERACT WITH APIS
AND PARSE DATA FORMATS IN PYTHON

Introduction

- What you'll learn
 - How to interact with APIs
 - Parsing JSON and XML data
 - Error handling in API requests
 - Live demonstration in PyCharm

What is an API?



API (Application Programming Interface)



Common Use cases

Fetching weather data (ie Open Mateo, OpenWeather API)

Social media feeds (ie Facebook, Twitter API)



Data formats: JSON & XML

JSON vs XML

Feature	JSON	XML
Readability	More compact, human-readable	Verbose, structured with tags
Parsing in Python	Easy (built-in support)	Requires XML parsing libraries
Usage	Web APIs, lightweight data exchange	Configuration files, complex data structures

Making API Requests in Python

Requests
Library

Making a GET
Request



Handling API Response Codes & Errors

- Why handle errors?
- Common HTTP status codes:
 - 200 ok – request successful
 - 404 not found – incorrect URL
 - 500 internal server error – server issues

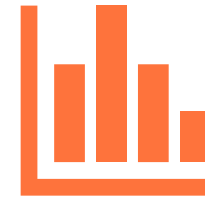
Working with JSON Data



Structure of JSON:
Key-value pairs



Parsing JSON in
Python



Extracting specific
data

Working with XML Data

Why XML

Parsing XML using
ElementTree



Live Demo

- Steps
 - Make an API request
 - Parse JSON response
 - Handle Errors
 - Parse XML response (if available) Live Coding: call OpenMateo API and parse JSON response in PyCharm

Best Practices for Handling API Data



Always check response codes before processing data



Use try-except blocks to handle unexpected errors



Rate limits: some APIs limit requests; respect their guidelines



API keys: many APIs require authentication; store keys securely
